EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 7-IDA: IDA Pro — Read, Hack, and Patch Constants and a 1602 LCD String (Raw `.bin`)

ARM Cortex-M33 • GDB • Ghidra

October 20, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the lesson project with `Release` and get both an `.elf` and a raw `.bin`
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load the raw `.bin` into IDA at `0x10000000`
- Read how a `#define` macro **and** a `const` variable both fold to bare instruction immediates
- Tell a 16-bit `movs r1, #42` from a 32-bit `movw r1, #1337` in the disassembly
- Follow the I2C path through the `i2c_inst_t` struct: `I2C_PORT` -> `i2c1` -> `&i2c1_inst` -> `hw` -> `0x40098000`
- Optionally **hack the LCD string live** by pointing `r0` at a RAM replacement
- **Resolve the functions in the IDA GUI** using the ELF symbol map, including the `lcd_1602.c` driver symbols
- **Patch** both constants (`42 -> 43` , `1337 -> 1344`) and the LCD string (`"Reverse" -> "Exploit"`)
- **Export** the patched image, convert it to UF2, and export it

How This Guide Works

The build produces two files for the project:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table	Ground truth for every function address and name
`.bin`	The raw flash image, no headers, no symbols	The image we load into IDA and reverse

The `.bin` is built **from** the `.elf` , so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in IDA, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the Week 7 lesson, and the Week 7 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it **inlines** the `static` helpers `init_i2c_and_lcd` and `write_lcd_greeting` (and the whole `lcd_1602.c` static helper chain) straight into `main` or into the public `lcd_*` functions, and it folds both `FAV_NUM` and `OTHER_FAV_NUM` down to immediate values. If you build `Debug`, the SDK function addresses move and the helpers stay separate calls, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**:

d prove what the code does. ge. 3. Resolve the functions in IDA using the ELF symbol map. 4. Patch the bytes, export, convert and export.

Project	Serial output	Also does	The hacks
`0x0017_constants`	`FAV_NUM: 42`, `OTHER_FAV_NUM: 1337`	I2C1 (SDA GP2, SCL GP3) drives a 1602 LCD, writing `Reverse` / `Engineering`	`42 -> 43`, `1337 -> 1344`, and `"Reverse" -> "Exploit"`

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

> **The surprise of this week is that `const` is not in memory.** `#define FAV_NUM 42` becomes the 16-bit `movs r1, #42`; `const int OTHER_FAV_NUM = 1337` *also* becomes an immediate, the 32-bit `movw r1, #1337`. The compiler only keeps a `const` in `.rodata` if the program **takes its address** (`&OTHER_FAV_NUM`); this program never does, so the `const` is inlined exactly like the macro. You patch an instruction operand, not a data word.

Part 1: Build and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **IDA Pro** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the project with `Release`

Run this inside `0x0017_constants/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point IDA at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so IDA never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the IDA console, so the whole build -> patch -> flash loop stays inside IDA. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
proj = os.path.join(root, "0x0017_constants")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x0017_constants")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x0017_constants")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

The build directory now contains the pair we need:

- `0x0017_constants/build/0x0017_constants.elf` and `.bin` — the `.bin` is **17980** bytes (`0x463c`)

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...` :

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`~/.pico-sdk/toolchain/14_2_Rel1/bin`
Linux x64	`/usr`

> **This guide's toolchain lives at** `~/.pico-sdk/toolchain/14_2_Rel1/bin`. All of `arm-none-eabi-nm`, `arm-none-eabi-objdump`, and `arm-none-eabi-gdb` resolved in this document come from there. If your install is elsewhere, `which arm-none-eabi-nm` tells you where to point.

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on the ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x0017_constants.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x0017_constants.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T` / `t` type is a function. Here are the functions this lesson uses. The signatures come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"`, so they are exact.

Our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (`init_i2c_and_lcd` and `write_lcd_greeting` inlined)

Our I2C/LCD driver (`lcd_1602.c`) and the SDK I2C functions `main` reaches:

Address	ELF symbol	Signature	Role
`0x100002bc`	`lcd_i2c_init`	`void lcd_i2c_init(i2c_inst_t*, uint8_t, int, uint8_t)`	store config + HD44780 reset/configure

Address	ELF symbol	Signature	Role
`0x100006f4`	`lcd_set_cursor`	`void lcd_set_cursor(int, int)`	move the HD44780 cursor
`0x100007f0`	`lcd_puts`	`void lcd_puts(const char*)`	write a string to the LCD
`0x10003cdc`	`i2c_init`	`uint i2c_init(i2c_inst_t*, uint)`	SDK I2C init (100 kHz)
`0x10003d28`	`i2c_write_blocking`	`int i2c_write_blocking(i2c_inst_t*, uint8_t, const uint8_t*, size_t, bool)`	one blocking I2C transfer
`0x100008f0`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select (I2C pins)
`0x1000092c`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`	SDK pull config (`gpio_pull_up` inlined)

> The `lcd_1602.c` static helpers — `pcf_write_byte`, `pcf_pulse_enable`, `lcd_write4`, `lcd_send`, `lcd_store_config`, `lcd_hd44780_reset`, `lcd_hd44780_configure` — have **no symbol of their own** in the `Release` build. They are inlined into `lcd_i2c_init`, `lcd_set_cursor`, and `lcd_puts`, which is why those three functions are large and call `i2c_write_blocking` and the `sleep_*` helpers directly.

The stdio/UART and printf chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x10001368`	`sleep_us`	`void sleep_us(uint64_t)`	SDK microsecond delay
`0x10001440`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay
`0x10001624`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock
`0x10001638`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x100016b8`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x1000188c`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART clock lookup
`0x100035a4`	`exit`	`void exit(int)`	C runtime exit
`0x100035ac`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x100035d8`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x100036e8`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output

Address	ELF symbol	Signature	Role
`0x100037d4`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x100037fc`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x100038c8`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x1000398c`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper
`0x10003b48`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x10003e24`	`strlen`	`size_t strlen(const char*)`	C runtime string length

SDK helpers the `printf` path reaches:

Address	ELF symbol	Signature	Role
`0x10003548`	`vfctprintf`	`int vfctprintf(void (*)(char, void*), void*, const char*, va_list)`	printf format engine
`0x10002b64`	`_vsnprintf`	`int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)`	the format dispatcher
`0x10001f48`	`_ntoa_format`	`unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)`	number formatter
`0x10001eac`	`_out_rev`	`unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)`	reversed-digit output
`0x1000211c`	`_out_char`	`void _out_char(char, void*, size_t, size_t)`	single-char sink

Two things in this project have **no symbol of their own**, because the compiler inlined them into `main`:

- `init_i2c_and_lcd` and `write_lcd_greeting` — the two `static` helpers in our own source are inlined, so there is no address to rename. You see their bodies directly inside `main`.
- `gpio_pull_up` — `static inline` in the SDK, so `gpio_pull_up(2)` and `gpio_pull_up(3)` compile to direct calls to `gpio_set_pulls` at `0x10000258` and `0x10000262`.

Part 2: Load the Raw `.bin` into IDA

Start from a fresh IDA state. If you already have a database for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into IDA

A raw `.bin` has no headers, so IDA cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, IDA may load it with a guessed architecture, and every address in this lesson will be wrong.

1.1 Open the file

1. **File ▸ Open...** (Ctrl/Cmd+O).
2. Select the `.bin` file and click **Open**.

IDA opens the **"Load a new file"** dialog because a raw `.bin` has no header.

1.2 Set the format and processor

In the *Load a new file* dialog:

1. **Format**: leave it as **Binary file** (IDA auto-detects a headerless image).
2. **Processor type**: click the **...** button next to the processor field. In the *Processor type* dialog:
    - Family: **ARM**
    - Type: **ARM Little-endian** (the 32-bit ARM module).
    - Click **OK**.

> If IDA asks **"Do you want to disassemble it as 64-bit code?"** — click **No**. The RP2350 is 32-bit; answering *Yes* loads AArch64 and you get `X` registers / data. (This is the single most common cause of the `DCB` wall.)

1.3 Configure the ARM architecture and Thumb mode

Still in the *Load a new file* dialog, open the processor options (the **Processor options** / **Set...** button, which opens the dialog titled **ARM architecture options**):

- **ARM architecture**: choose **ARMv8-M** (this is the Cortex-M33 / RP2350 core).
- **Thumb instructions**: select **Thumb-2** (ARMv8-M is Thumb-only, so this is the correct mode).
- Leave **automatic ARM-THUMB switch** off; we want Thumb throughout.

1.4 Set the loading address

- **Loading address**: enter `10000000` (hex; the RP2350 XIP flash base). IDA rejects addresses that don't look like ROM/RAM with *"The loading address should belong to RAM or ROM"* — `0x10000000` is the correct flash window.

Leave **Manual load** and **Create segments** at their defaults. Click **OK**.

1.5 Force Thumb, then seed the code from the vector table

A raw `.bin` has no header, so IDA's loader has **no entry point** and defaults the segment to **ARM mode** (which cannot decode Thumb). Two required steps — in this order:

A. Set the segment to Thumb (do this first).

- **Edit ▸ Segments ▸ Set default segment register value...** → register `T` = `1` (or press `Alt+G`). IDA reanalyzes the segment as Thumb automatically.

B. Seed the code from the vector table.

1. **G** → `10000000`. The first two words are the initial stack pointer and the **reset vector**.
2. **Read the 4-byte word at** `0x10000004` (do **not** assume its value — it differs per build).
3. **Clear bit 0** (the ARM **Thumb** flag) → the reset handler address.
4. **G** to that address → press **C** (**MakeCode**). IDA disassembles the reset handler and follows its `bl`s into `main`.

Once `main` shows code, the import is correct.

Why IDA needs this and others don't: IDA's generic raw-binary ARM loader treats the image as flat data and waits for you to set Thumb and mark the entry point. It's an IDA quirk, not a mistake on your part.

Step 8: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Static — Resolve the Functions in IDA and Patch

Step 16: Resolve the functions in the IDA GUI

We now name the functions in IDA using the ELF symbol map from Step 4. IDA loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	IDA action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in IDA (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_100037fc()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

The decompiler header then shows the real prototype, and calls to the function read cleanly instead of `sub_<addr>()`. `N` is only for renaming without touching the type; `Y` alone sets both the name and the type.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. IDA parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The view jumps there; the cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter. That sets the name to `main` and the type to `int(void)`.

> **IDA shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and IDA's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `i2c_init`

1. `G` -> `0x10003cdc`.
2. `Y` -> `uint i2c_init(i2c_inst_t* i2c, uint baudrate)`.

> **`i2c_init` returns `uint`, not `void`.** The SDK's `i2c_init` returns the actual configured baud rate; the ELF says `unsigned int (i2c_inst_t *, uint)`. Keep the return type.

Worked example: `lcd_i2c_init`

1. `G` -> `0x100002bc`.
2. `Y` -> `void lcd_i2c_init(i2c_inst_t* i2c, uint8_t pcf_addr, int nibble_shift, uint8_t backlight_mask)`.

This is our own `lcd_1602.c` code. In this build it is one big function: the compiler inlined `lcd_store_config`, `lcd_hd44780_reset`, and `lcd_hd44780_configure` into it.

Worked example: `lcd_set_cursor`

1. `G` -> `0x100006f4`.
2. `Y` -> `void lcd_set_cursor(int line, int position)`.

Worked example: `lcd_puts`

1. `G` -> `0x100007f0`.
2. `Y` -> `void lcd_puts(const char* s)`.

Worked example: `gpio_set_function`

1. `G` -> `0x100008f0`.
2. `Y` -> `void gpio_set_function(uint gpio, gpio_function_t fn)`.

Worked example: `gpio_set_pulls`

1. `G` -> `0x1000092c`.
2. `Y` -> `void gpio_set_pulls(uint gpio, bool up, bool down)`.

This is what `gpio_pull_up(2)` in our source compiles to: the SDK's `static inline` `gpio_pull_up` disappears, and `main` calls `gpio_set_pulls(2, true, false)` directly at `0x10000258`.

Worked example: `__wrap_printf`

1. `G` -> `0x1000398c`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

Worked example: `stdio_init_all`

1. `G` -> `0x100037fc`.
2. `Y` -> `bool stdio_init_all(void)`.

> **`printf` in our source is `__wrap_printf` in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper, which forwards to `__wrap_vprintf`. Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK. `main` calls `stdio_init_all`, `i2c_init`, `gpio_set_function`, `gpio_set_pulls`, `lcd_i2c_init`, `lcd_set_cursor`, `lcd_puts`, and `printf`, so we follow that chain down.

The call chain for this project:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init, stdio_set_driver_enabled
│                                        └── uart_init ── clock_get_hz, busy_wait_us
├── i2c_init
├── gpio_set_function
├── gpio_set_pulls                (gpio_pull_up inlined)
├── lcd_i2c_init ── i2c_write_blocking, sleep_us, sleep_ms
│   └── pcf_write_byte / pcf_pulse_enable / lcd_write4 / lcd_send / lcd_store_config /
│       lcd_hd44780_reset / lcd_hd44780_configure   (all inlined; no calls)
├── lcd_set_cursor ── i2c_write_blocking, sleep_us
├── lcd_puts ── i2c_write_blocking, sleep_us
└── __wrap_printf ── __wrap_vprintf ── vfctprintf ── _vsnprintf
    │                                              └── _ntoa_format / _out_rev / _out_char
    ├── time_us_64
    └── stdio_out_chars_crlf
```

Resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`

Address	Rename to (N)	Signature (Y)
0x100002bc	lcd_i2c_init	void lcd_i2c_init(i2c_inst_t*, uint8_t, int, uint8_t)
0x100006f4	lcd_set_cursor	void lcd_set_cursor(int, int)
0x100007f0	lcd_puts	void lcd_puts(const char*)
0x100008f0	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x1000092c	gpio_set_pulls	void gpio_set_pulls(uint, bool, bool)
0x10001368	sleep_us	void sleep_us(uint64_t)
0x10001440	sleep_ms	void sleep_ms(uint32_t)
0x10001624	time_us_64	uint64_t time_us_64(void)
0x10001638	busy_wait_us	void busy_wait_us(uint64_t)
0x100016b8	uart_init	uint uart_init(uart_inst_t*, uint)
0x1000188c	clock_get_hz	unsigned long clock_get_hz(clock_handle_t)
0x100035a4	exit	void exit(int)
0x100035ac	runtime_init	void runtime_init(void)
0x100035d8	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x100036e8	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x100037d4	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x100037fc	stdio_init_all	bool stdio_init_all(void)
0x100038c8	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x1000398c	__wrap_printf	int __wrap_printf(const char*, ...)
0x10003b48	stdio_uart_init	void stdio_uart_init(void)
0x10003cdc	i2c_init	uint i2c_init(i2c_inst_t*, uint)
0x10003d28	i2c_write_blocking	int i2c_write_blocking(i2c_inst_t*, uint8_t, const uint8_t*, size_t, bool)
0x10003e24	strlen	size_t strlen(const char*)

Resolve the SDK helpers the `printf` path reaches:

Address	Rename to (N)	Signature (Y)
0x10003548	vfctprintf	int vfctprintf(void (*)(char, void*), void*, const char*, va_list)
0x10002b64	_vsnprintf	int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)

Address	Rename to (N)	Signature (Y)
`0x10001f48`	`_ntoa_format`	`unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)`
`0x10001eac`	`_out_rev`	`unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)`
`0x1000211c`	`_out_char`	`void _out_char(char, void*, size_t, size_t)`

> **A `void` return type may not stick — here is the fix.** IDA treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type IDA thinks it can infer.
>
> Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:
>
> ```
> from binaryninja import ReturnValue, Type
> fn = bv.get_function_at(0x1000015c)
> if fn is not None:
>     fn.return_value = ReturnValue(Type.void())
> ```
>
> That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

> `i2c1_inst` **is data, not a function.** `arm-none-eabi-nm -n` lists `2000062c T i2c1_inst`. The `T` is a **global data** symbol: the linker parks the `i2c1_inst` struct at RAM address `0x2000062c`. Its first word is the hardware pointer `0x40098000`, the I2C1 register base. There is no function there; do not `Y` it with a prototype.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into IDA's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
typedef void (*out_fct_type)(char, void*, size_t, size_t);
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
struct i2c_inst;
typedef struct i2c_inst i2c_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x100002bc: ("lcd_i2c_init",              "void lcd_i2c_init(i2c_inst_t*, uint8_t, int,
uint8_t)"),
    0x100006f4: ("lcd_set_cursor",            "void lcd_set_cursor(int, int)"),
    0x100007f0: ("lcd_puts",                  "void lcd_puts(const char*)"),
    0x100008f0: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x1000092c: ("gpio_set_pulls",            "void gpio_set_pulls(uint, bool, bool)"),
    0x10001368: ("sleep_us",                  "void sleep_us(uint64_t)"),
    0x10001440: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x10001624: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x10001638: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x100016b8: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x1000188c: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x100035a4: ("exit",                      "void exit(int)"),
    0x100035ac: ("runtime_init",              "void runtime_init(void)"),
    0x100035d8: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x100036e8: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x100037d4: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x100037fc: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x100038c8: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x1000398c: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x10003b48: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10003cdc: ("i2c_init",                  "uint i2c_init(i2c_inst_t*, uint)"),
    0x10003d28: ("i2c_write_blocking",        "int i2c_write_blocking(i2c_inst_t*, uint8_t, const
uint8_t*, size_t, bool)"),
    0x10003e24: ("strlen",                    "size_t strlen(const char*)"),
    0x10003548: ("vfctprintf",                "int vfctprintf(void (*)(char, void*), void*, const
char*, va_list)"),
    0x10002b64: ("_vsnprintf",                "int _vsnprintf(out_fct_type, char*, size_t, const
char*, va_list)"),
    0x10001f48: ("_ntoa_format",              "unsigned _ntoa_format(out_fct_type, char*, size_t,
size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)"),
    0x10001eac: ("_out_rev",                  "unsigned _out_rev(out_fct_type, char*, size_t,
size_t, const char*, size_t, unsigned, unsigned)"),
```

```
    0x1000211c: ("_out_char",                "void _out_char(char, void*, size_t, size_t)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)
```

SDK type names (`i2c_inst_t` , `uart_inst_t` , `stdio_driver_t` , `gpio_function_t` , plus `uint` , `va_list` , `clock_handle_t` , and `out_fct_type`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first (an opaque `struct` / `enum` / `typedef` is enough to parse). If you add a function that uses another SDK type, add a definition for it to that block too.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main` . Once the functions above are typed, it reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    i2c_init(&i2c1_inst, 0x186a0);              // i2c_init(i2c1, 100000)
    gpio_set_function(2, GPIO_FUNC_I2C);
    gpio_set_function(3, GPIO_FUNC_I2C);
    gpio_set_pulls(2, true, false);             // gpio_pull_up(2)
    gpio_set_pulls(3, true, false);             // gpio_pull_up(3)
    lcd_i2c_init(&i2c1_inst, 0x27, 4, 8);       // lcd_i2c_init(i2c1, 0x27, 4, 0x08)
    lcd_set_cursor(0, 0);
    lcd_puts("Reverse");
    lcd_set_cursor(1, 0);
    lcd_puts("Engineering");
    while (true) {
        __wrap_printf("FAV_NUM: %d\r\n", 0x2a);         // FAV_NUM  = 42
        __wrap_printf("OTHER_FAV_NUM: %d\r\n", 0x539);  // OTHER_FAV_NUM = 1337
    }
}
```

The `0x2a` and `0x539` are the constants we will patch. Both are **immediates in the instruction stream** — there is no `.rodata` word to change, which is why the patch edits the instruction operand. Now make the hacks permanent.

Step 18: Patch 1 — change `FAV_NUM` from 42 to 43

Go to `0x1000028e` :

```
1000028e: 2a 21   movs  r1, #42   @ 0x2a
```

The halfword is `0x212a` , stored little-endian as `2a 21` . The immediate is the low byte, so the byte at the instruction's own address is `0x2a` . Change it to `0x2b` (43).

Project	Address	Before	After	Effect
`0x0017`	`0x1000028e`	`2a`	`2b`	`movs r1, #42` -> `#43` , prints `FAV_NUM: 43`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x1000028e` and change the byte `2A` to `2B` .
4. Return to the linear view, right-click the function -> `Reanalyze` .

Option B — Python console:

```
bv.write(0x1000028e, b"\x2b")
print(hex(bv.read(0x1000028e, 1)[0]))  # -> 0x2b
```

After reanalysis the instruction reads `movs r1, #43` .

Step 18b: Patch 2 — change `OTHER_FAV_NUM` from 1337 to 1344

Go to `0x10000296` :

```
10000296: 40 f2 39 51   movw  r1, #1337   @ 0x539
```

This is the 32-bit Thumb-2 encoding of `movw r1, #0x539` . The four bytes and their roles:

```
+-------------------------------------------------------------------+
|  movw r1, #0x539  ->  bytes: 40 F2 39 51                          |
|                                                                   |
|  Byte 0: 0x40  -+                                                 |
|  Byte 1: 0xF2  -+   First halfword (opcode + upper imm bits)      |
|  Byte 2: 0x39  ---- Lower 8 bits of immediate (imm8) <- CHANGE    |
|  Byte 3: 0x51  ---- Destination register (r1) + upper imm bits    |
|                                                                   |
|  imm16 = 0x0539 = 1337 decimal                                    |
|  imm8 field = 0x39 (lower 8 bits of the value)                    |
|                                                                   |
+-------------------------------------------------------------------+
```

The imm8 byte is the **third** byte of the 4-byte instruction: the instruction starts at `0x10000296` , so the byte to change is `0x10000296 + 2 = 0x10000298` . Change `0x39` to `0x40` , which changes the value from `0x539` (1337) to `0x540` (1344).

Project	Address	Before	After	Effect
`0x0017`	`0x10000298`	`39`	`40`	`movw r1, #1337` -> `#1344` , prints `OTHER_FAV_NUM: 1344`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10000298` and change the byte `39` to `40` .
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10000298, b"\x40")
print(bv.read(0x10000296, 4).hex())  # -> 40f24051
```

> **Do not patch `0x10000296` itself.** That is the instruction's first byte (the opcode), not the immediate. The immediate's low 8 bits are at `0x10000298`; patching the opcode corrupts the instruction.

Step 18c: Patch 3 — change the LCD text from `"Reverse"` to `"Exploit"`

The string `"Reverse"` starts at `0x10003ee8`. Its eight bytes are `52 65 76 65 72 73 65 00` (`Reverse\0`). Change them to `45 78 70 6c 6f 69 74 00` (`Exploit\0`). **Both strings are exactly seven characters**, so the replacement fits without touching `"Engineering"` at `0x10003ef0`.

Project	Address	Before	After	Effect
`0x0017`	`0x10003ee8`	`52 65 76 65 72 73 65 00`	`45 78 70 6c 6f 69 74 00`	LCD line 1 prints `Exploit` instead of `Reverse`

ASCII reference:

Character	Hex
E	`0x45`
x	`0x78`
p	`0x70`
l	`0x6c`
o	`0x6f`
i	`0x69`
t	`0x74`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10003ee8` and change the eight bytes `52 65 76 65 72 73 65 00` to `45 78 70 6c 6f 69 74 00`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10003ee8, b"Exploit\x00")
print(bv.read(0x10003ee8, 8))  # -> b'Exploit\x00'
```

Keep the replacement exactly eight bytes. If you use a shorter string you must pad it and keep the terminating `\0`, or `lcd_puts` will run into the `"Engineering"` string that follows at `0x10003ef0`.

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x0017_constants", "build"), "0x0017_constants-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 17980 /.../build/0x0017_constants-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base IDA loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x463c` = 17980). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.
- Reading `seg.start` for `seg.data_length` bytes therefore grabs exactly the image.

Two gotchas this avoids:

- **No relative path.** IDA's Python console runs with a read-only working directory (inside the app bundle), so `open("0x0017_constants-h.bin", "wb")` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/` — no machine-specific path and no database needed.
- **Read the image, not the whole view.** `bv.read(bv.start, bv.length)` spans the entire mapped range, which is not the image. The segment's `data_length` is the image size.

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0017_constants-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0017_constants-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the IDA console** — it is a normal Python interpreter, so you never have to leave the app. `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x0017_constants", "build"))   # the project build dir (writable)
> sys.argv = ["uf2conv.py", "0x0017_constants-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")   # path to your uf2conv.py
> ```
>
> This writes `hacked.uf2` next to the `.bin`, ready to drag onto the Pico.

Static stopping point

The patched image has been exported and converted to UF2. This IDA guide ends at static verification; it does not connect to, flash, or debug a live target.

Glossary

Term	Definition
AAPCS	ARM Architecture Procedure Call Standard — `r0`-`r3` for the first four arguments, `r0` for the return value
`.bss`	Section for uninitialized (or zero-initialized) static/global variables; zeroed by startup code
`const`	A source-level "read-only" qualifier; the compiler may still inline it as an immediate
`.data`	Section for initialized static/global variables; copied from flash to SRAM at boot
`#define`	Preprocessor text replacement performed before compilation; consumed by the compiler as a literal
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
`imm8`	The low 8 bits of a `movw` immediate, stored in the third byte of the 32-bit instruction
Immediate value	A constant embedded directly in an instruction, not fetched from memory
I2C	Inter-Integrated Circuit — a two-wire (SDA/SCL) serial bus; open-drain, needs pull-ups
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`
`movs`	16-bit Thumb move that loads an 8-bit immediate (0-255)
`movw`	32-bit Thumb-2 "move wide" that loads any 16-bit immediate (0-65535)
Open-drain	An output that can only pull a line LOW, not drive it HIGH; pull-ups restore HIGH
PCF8574	The I2C I/O expander on a typical 1602 LCD backpack; commonly at `0x27`
`.rodata`	Read-only section for constants and string literals; stays in flash
SCL / SDA	I2C Serial Clock and Serial Data lines
Struct	A user-defined type that groups related variables; the SDK uses one per I2C controller
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
`typedef`	Creates an alias for a type (for example `typedef struct i2c_inst i2c_inst_t`)
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

---

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Verify the static behavior from the recovered instructions.c`symbols), then patch the bytes –` 0x2a -> 0x2b`,` 0x39 -> 0x40`, and the eight-byte LCD string — and export the patched artifact.